



IIC3670 Procesamiento de Lenguaje Natural

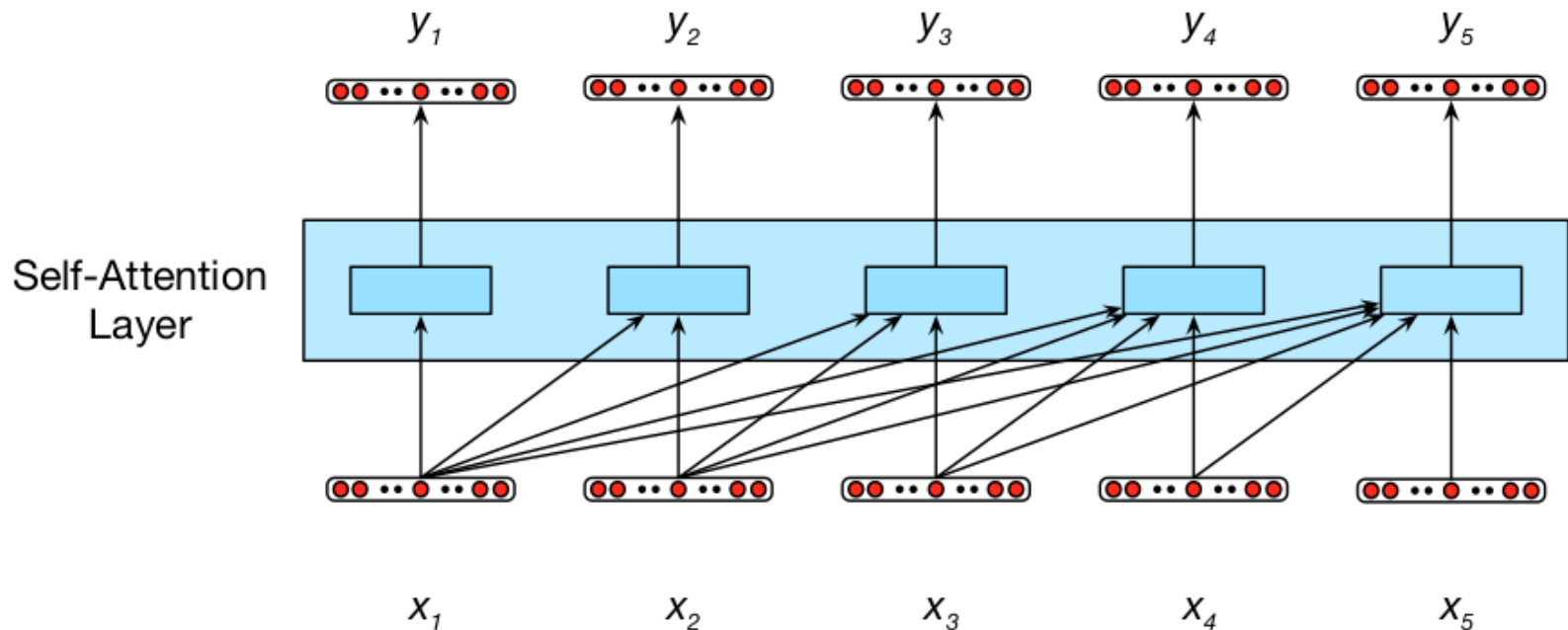
<https://github.com/marcelomendoza/IIC3670>

- TRANSFORMER -

Mecanismo de auto-atención

La auto-atención permite a la red extraer información desde contextos de largo variable sin la necesidad de pasar por una red recurrente.

En una capa de auto-atención, el modelo (FFNN) tiene acceso a todas las entradas procesadas hasta ese step:



Este enfoque se llama 'masked self-attention'.

Mecanismo de auto-atención

La base del mecanismo está en la comparación de un ítem de interés con otros de manera que revele su relevancia en el contexto del step actual.

La forma más simple de comparación es el producto punto de vectores. Por ejemplo, para calcular y_3 debemos comparar $x_3 \cdot x_1$, $x_3 \cdot x_2$ y $x_3 \cdot x_3$. Los productos se normalizan con una softmax para crear un vector de pesos α_{ij} , que indica la relevancia de cada entrada (j) al símbolo de posición i .

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

Mecanismo de auto-atención

Los Transformers agregan a la capa de self-attention parámetros adicionales que operan sobre los embeddings de entrada. La motivación se basa en tres roles que juegan los embeddings de entrada en el proceso de atención:

- Query: el input embedding se considera como foco actual de atención al compararse con las entradas precedentes.
- Key: En su rol como entrada precedente, el input embedding del step anterior se compara con el siguiente.
- Valor: se usa el input embedding para calcular el valor de salida en el time step actual.

Los parámetros que se agregan son:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

En el Transformer original, el input embedding es de dim=1024 pero se reducen a 64 al pasarlos por las matrices W (1024 x 64).

Mecanismo de auto-atención

Los Transformers en lugar de ingestar los input embeddings en la capa de self-attention, usan el vector q_i para el símbolo de step i , y lo comparan con los vectores k_j precedentes:

$$\mathbf{head}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

$\nearrow score(x_i, x_j) = q_i \cdot k_j$
 $\searrow$ Vector value

Los coeficientes de atención se calculan según:

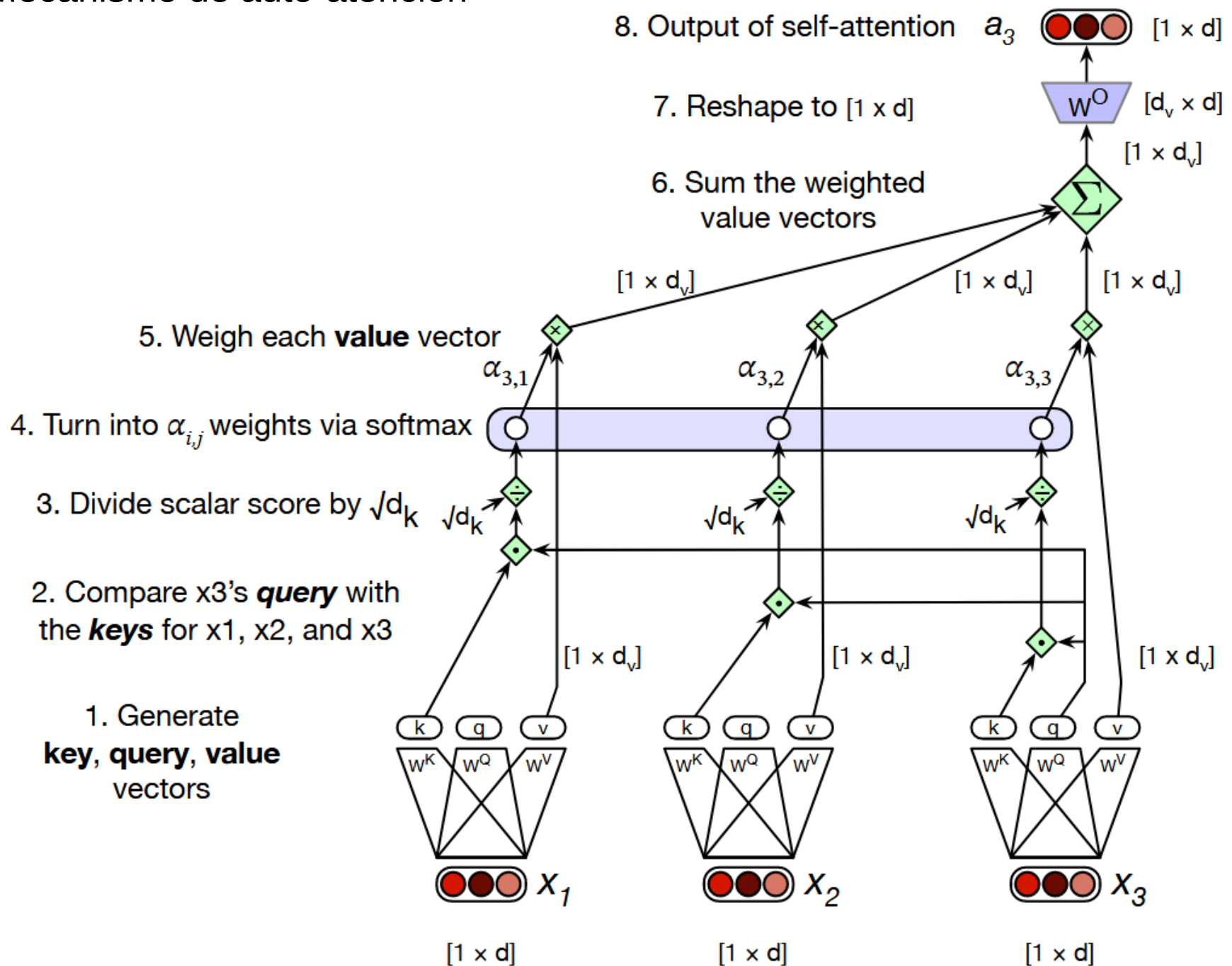
$$score(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(score(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

Y la salida se produce desde el head a una matriz de transferencia:

$$\mathbf{a}_i = \mathbf{head}_i \mathbf{W}^O$$

Mecanismo de auto-atención



Mecanismo de auto-atención

Auto-atención en el **encoder** (query, key y value):

Input

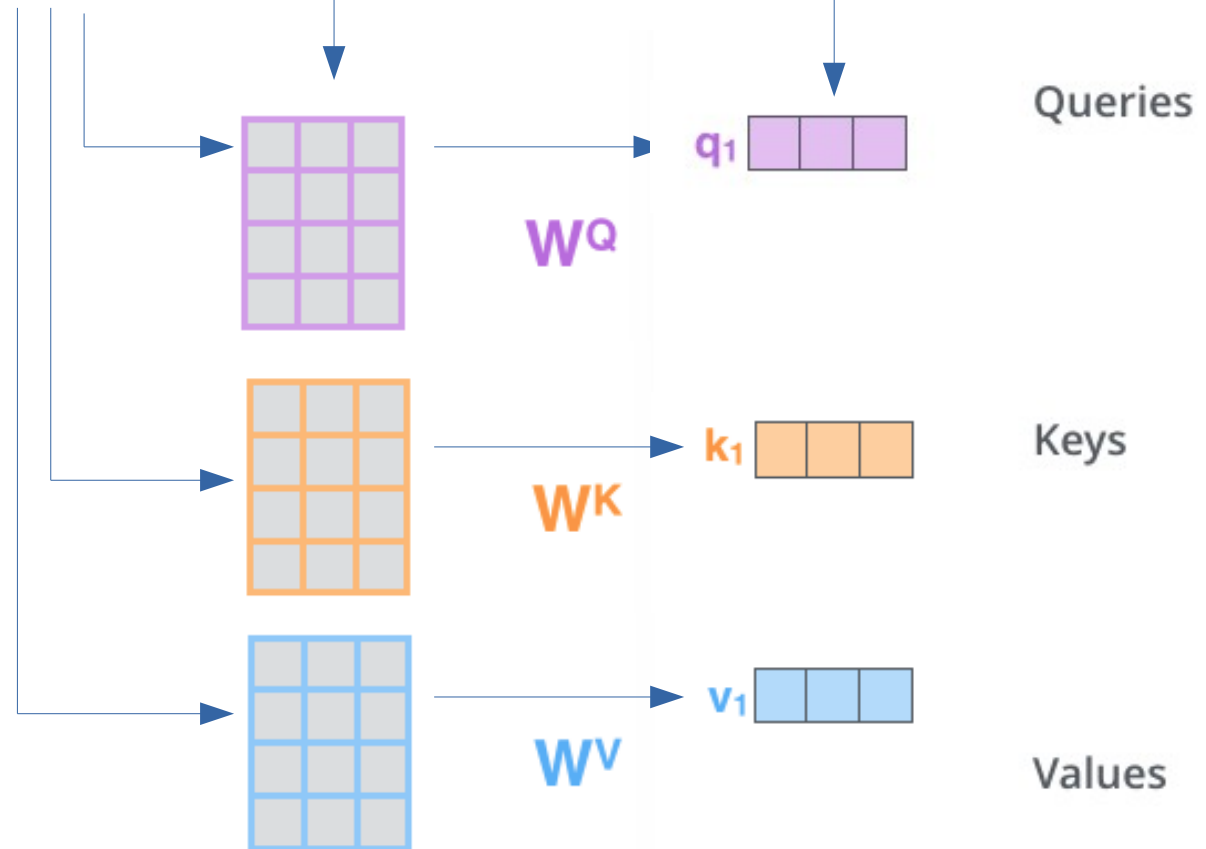
Thinking

Embedding

X_1

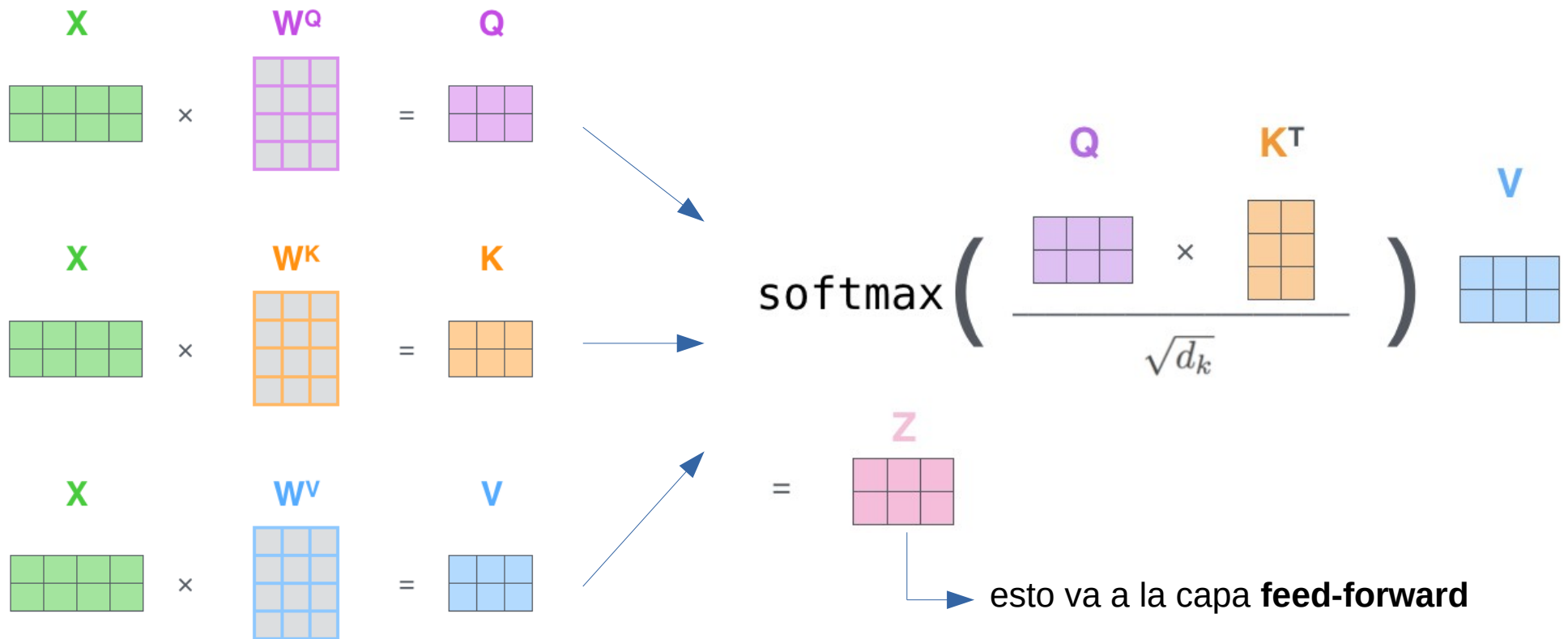
W^Q , W^K y W^V se aprenden durante el entrenamiento

estos vectores son de baja dimensionalidad



Mecanismo de auto-atención

Auto-atención en el **encoder** (cálculo matricial):



Mecanismo de auto-atención multi-cabecal

Los Transformers usan un mecanismo de auto-atención multi-cabecal para proporcionar al modelo la habilidad de aprender distintos aspectos de las relaciones entre entradas al mismo nivel de abstracción.

Cada cabezal tiene su propio set de parámetros q , k y v para proyectar la entrada a la capa correspondiente. La salida de una multi-head attention layer con A cabezales es de A veces la dimensionalidad de la entrada. Luego, los A vectores son combinados y reducidos a la dimensionalidad original usando una capa de proyección lineal:

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}^c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}^c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}^c}; \quad \forall c \quad 1 \leq c \leq A$$

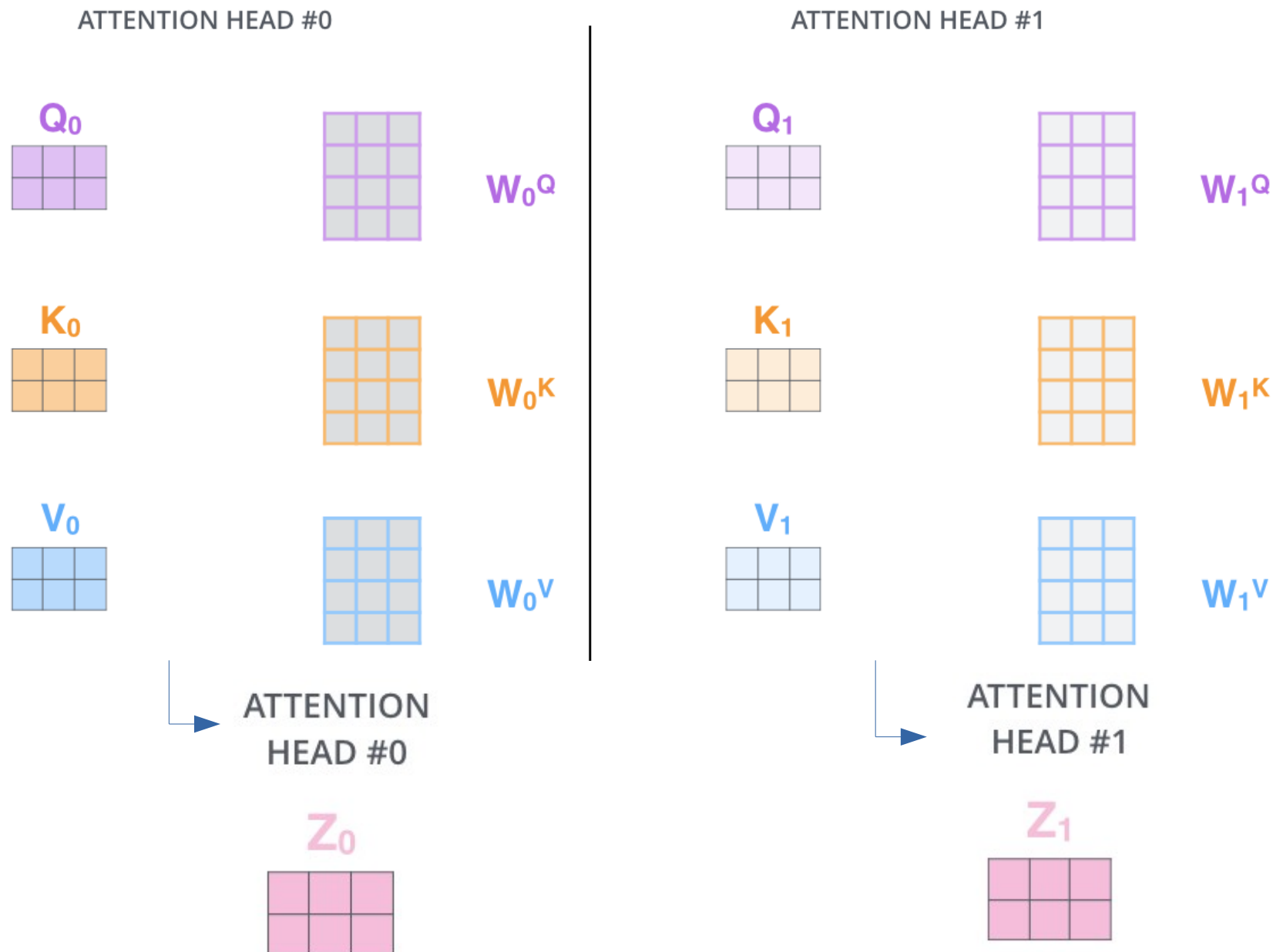
$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

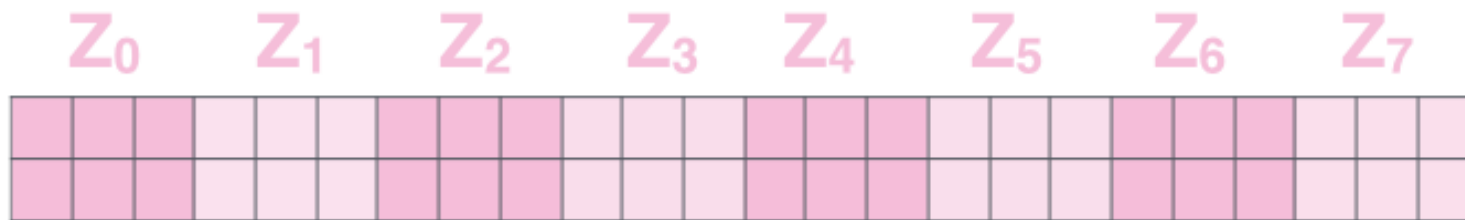
$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^A) \mathbf{W}^O$$

Mecanismo de auto-atención multi-cabezal

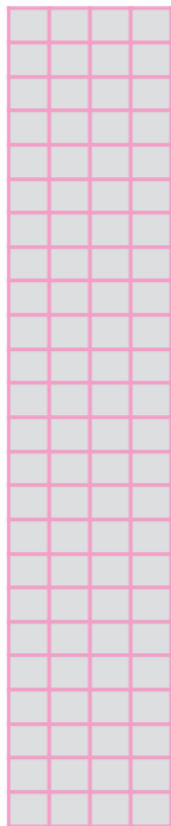


Mecanismo de auto-atención multi-cabezal

Concatenamos los Z ...



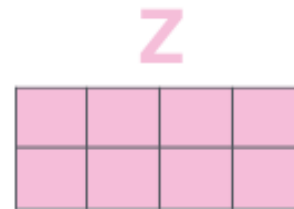
X



W^O

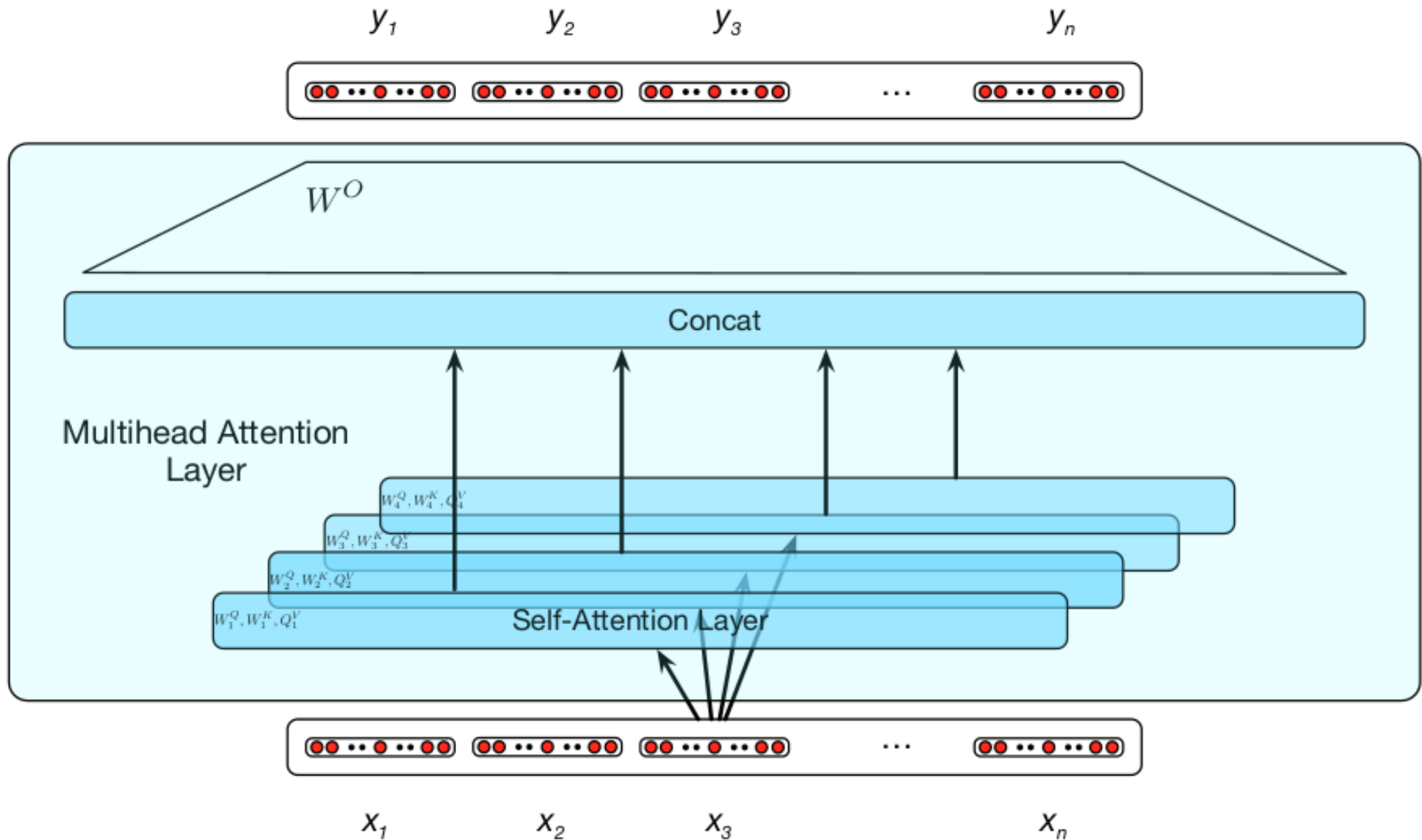
... y la reducimos a la dimensionalidad original.

$\longrightarrow =$



esto va a la capa **feed-forward**

Mecanismo de auto-atención multi-cabezal



Layer Norm

El transformer agrega mas elementos. Uno de ellos es LayerNorm.

LayerNorm debe escalar y trasladar cada vector de manera de evitar problemas con los cálculos de los gradientes. En rigor, LayerNorm se define por la siguiente función de transferencia:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

El tipo de normalización que se hace es una estandarización sobre cada vector:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$
$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

Gamma y beta se definen como parámetros aprendibles.

Conexiones residuales

Algo que viene de arquitecturas más antiguas y que ha mostrado buenos resultados cuando tienes muchas capas en una red neuronal es usar conexiones residuales, esto es, sumar la entrada a la salida de cada capa.

Además el transformer incluye una capa feedforward, a la salida del bloque. Colocando todos estos elementos tenemos las ecuaciones de transferencia del transformer a nivel de token:

$$\mathbf{t}_i^1 = \text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_j]), \quad j \leq i$$

$$\mathbf{t}_i^2 = \text{LayerNorm}(\mathbf{t}_i^1)$$

$$\mathbf{t}_i^3 = \mathbf{t}_i^2 + \mathbf{x}_i$$

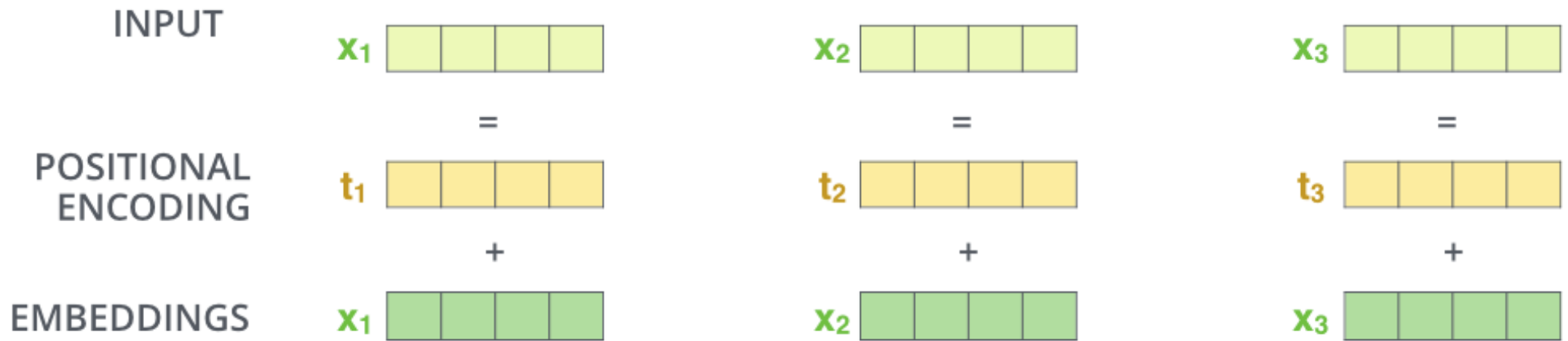
$$\mathbf{t}_i^4 = \text{FFN}(\mathbf{t}_i^3)$$

$$\mathbf{t}_i^5 = \text{LayerNorm}(\mathbf{t}_i^4)$$

$$\mathbf{y}_i = \mathbf{t}_i^5 + \mathbf{t}_i^3$$

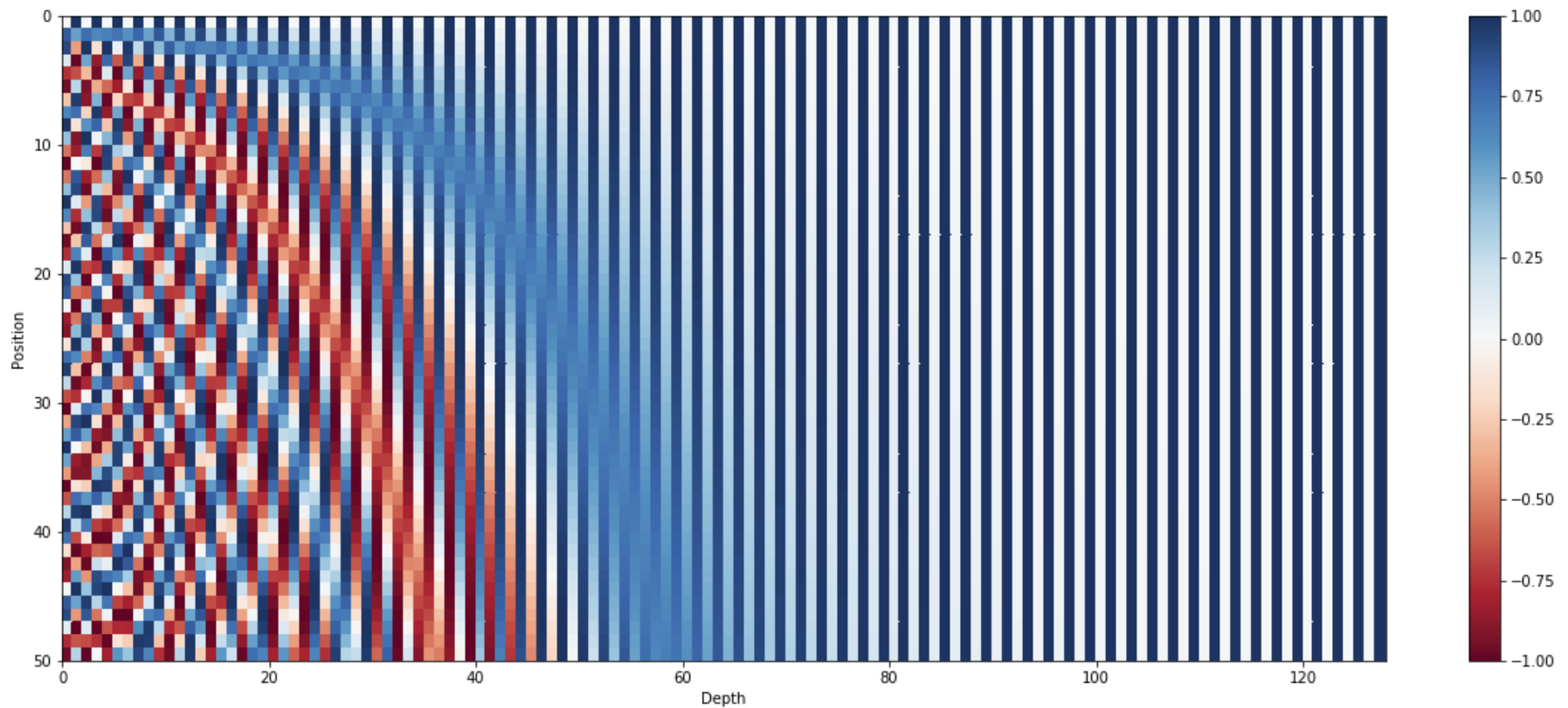
Positional encoding

Se suma un vector a la entrada, el cual codifica la posición de la palabra en el texto



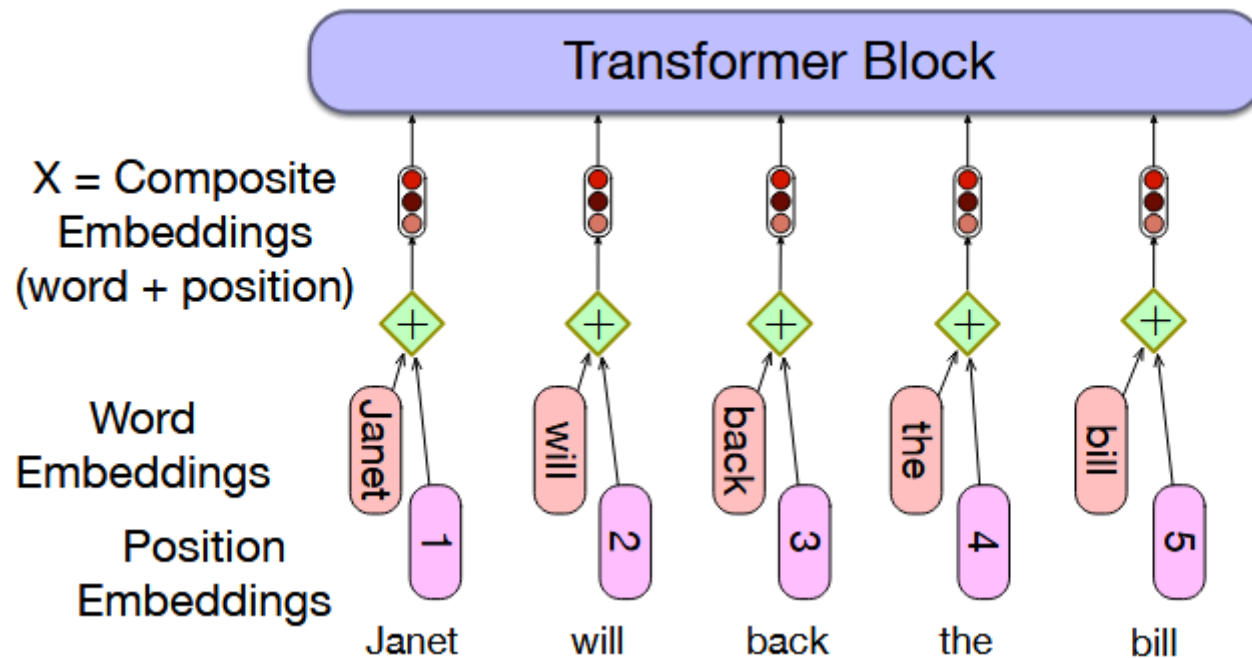
Positional encoding

Los positional encodings son estáticos:

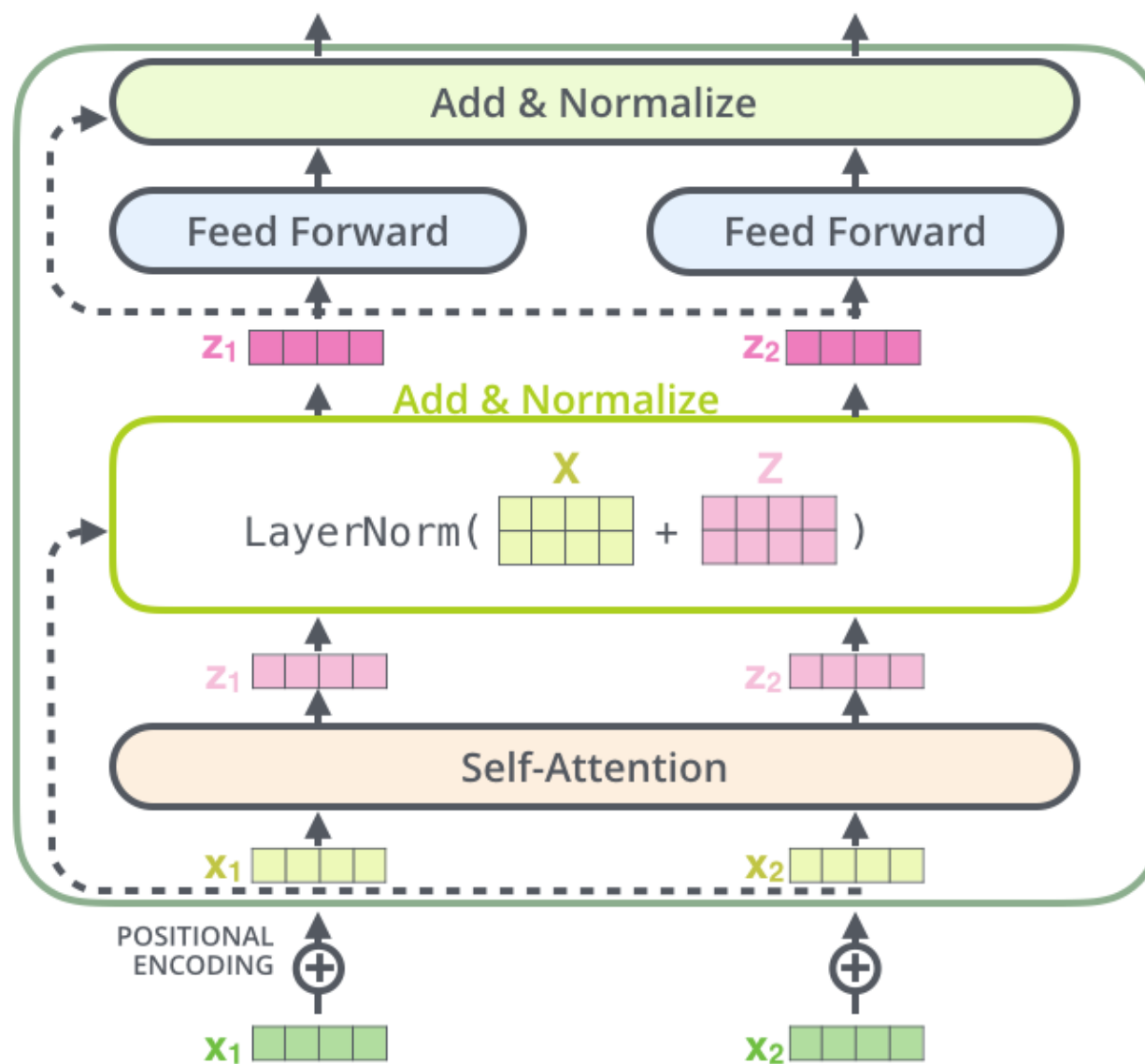


Positional encoding

Y los sumamos a la entrada del transformer block:



Finalmente tenemos el transformer block



Variaciones de la atención usando máscaras

Para controlar a que le prestamos atención, usamos una máscara en el producto entre q y k:

$$\mathbf{head} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{v}$$

$$\mathbf{A} = \mathbf{head} \mathbf{W}^O$$

En atención causal la máscara es (GPT):

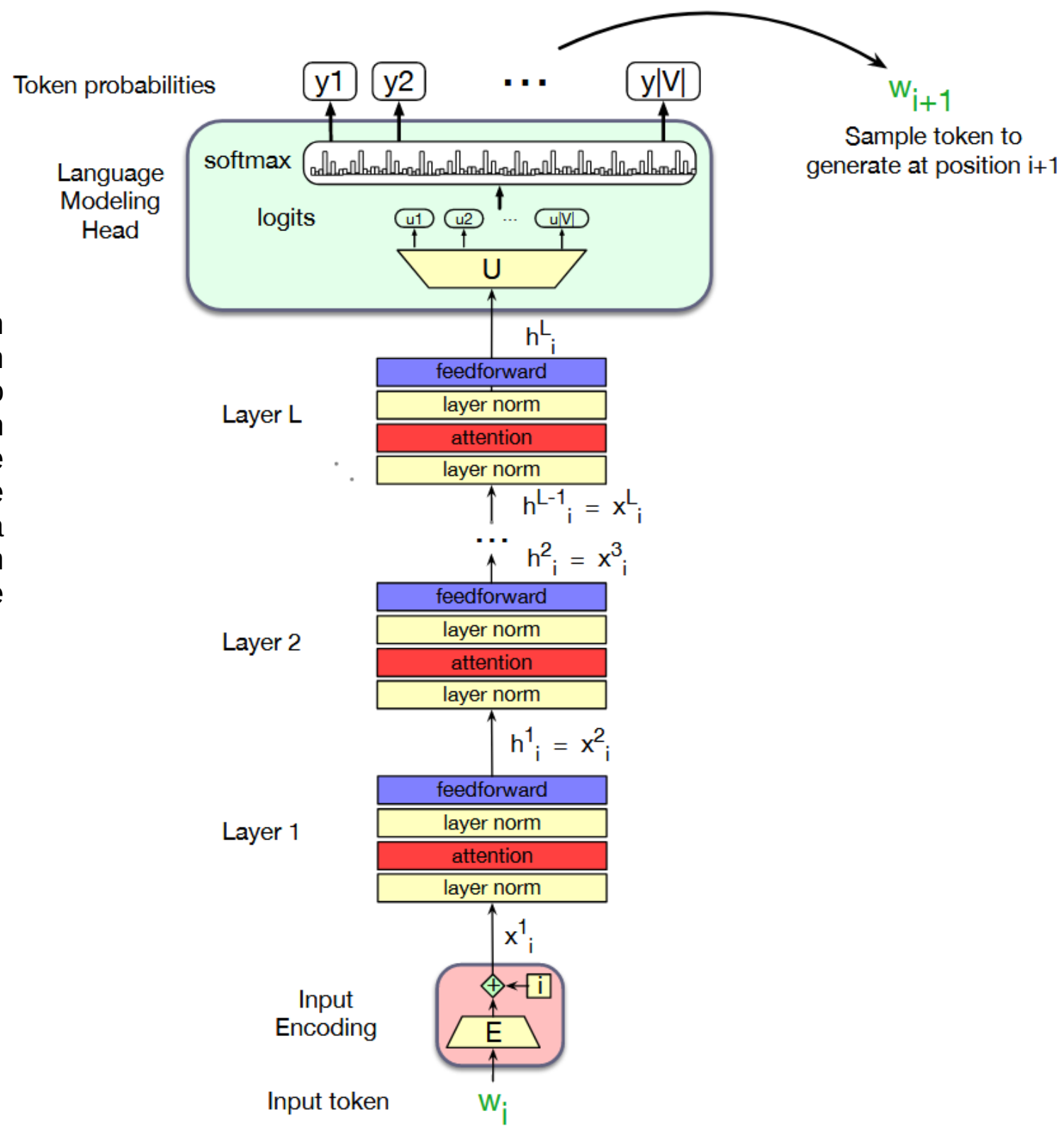
N	q1•k1	−∞	−∞	−∞
	q2•k1	q2•k2	−∞	−∞
	q3•k1	q3•k2	q3•k3	−∞
	q4•k1	q4•k2	q4•k3	q4•k4
		N		

Pero podemos poner atención bidireccional con esta máscara (BERT):

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4
		N		

Transformer decoder

Usa atención causal. Con next token prediction preentrenamos un modelo de lenguaje colocando en el top una softmax sobre el vocabulario. En este ejemplo se varía la posición de LayerNorm (primeras versiones de GPT):



- BERT -

Bidirectional Encoder Representations on Transformers (BERT)

- Entrada: WPM


$$\text{tokenize}(\text{text}_i) = x_i = (x_i^1, x_i^2, \dots, x_i^{n_i})$$

Word-piece model →

- **Word:** Jet makers feud over seat width with big orders at stake
- **wordpieces:** _J et _makers _fe ud _over _seat _width _with _big _orders _at _stake

LM que minimiza el
número de tokens
necesarios para
representar un
corpus



Basado en Byte-Pair
encoding (BPE)



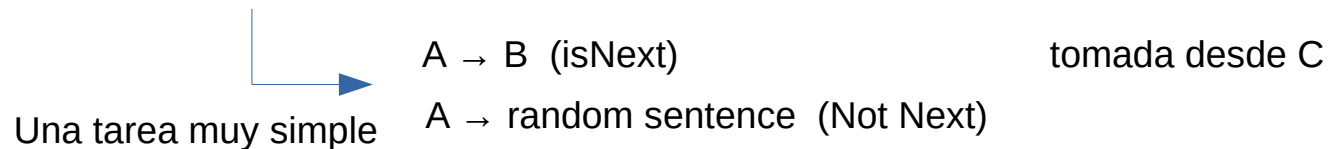
Reemplaza un 2-
char gram frecuente
por un 1-gram poco
usado

Bidirectional Encoder Representations on Transformers (BERT)

- Usa contexto bidireccional (atención bidireccional) en el transformer

Building blocks de **BERT**:

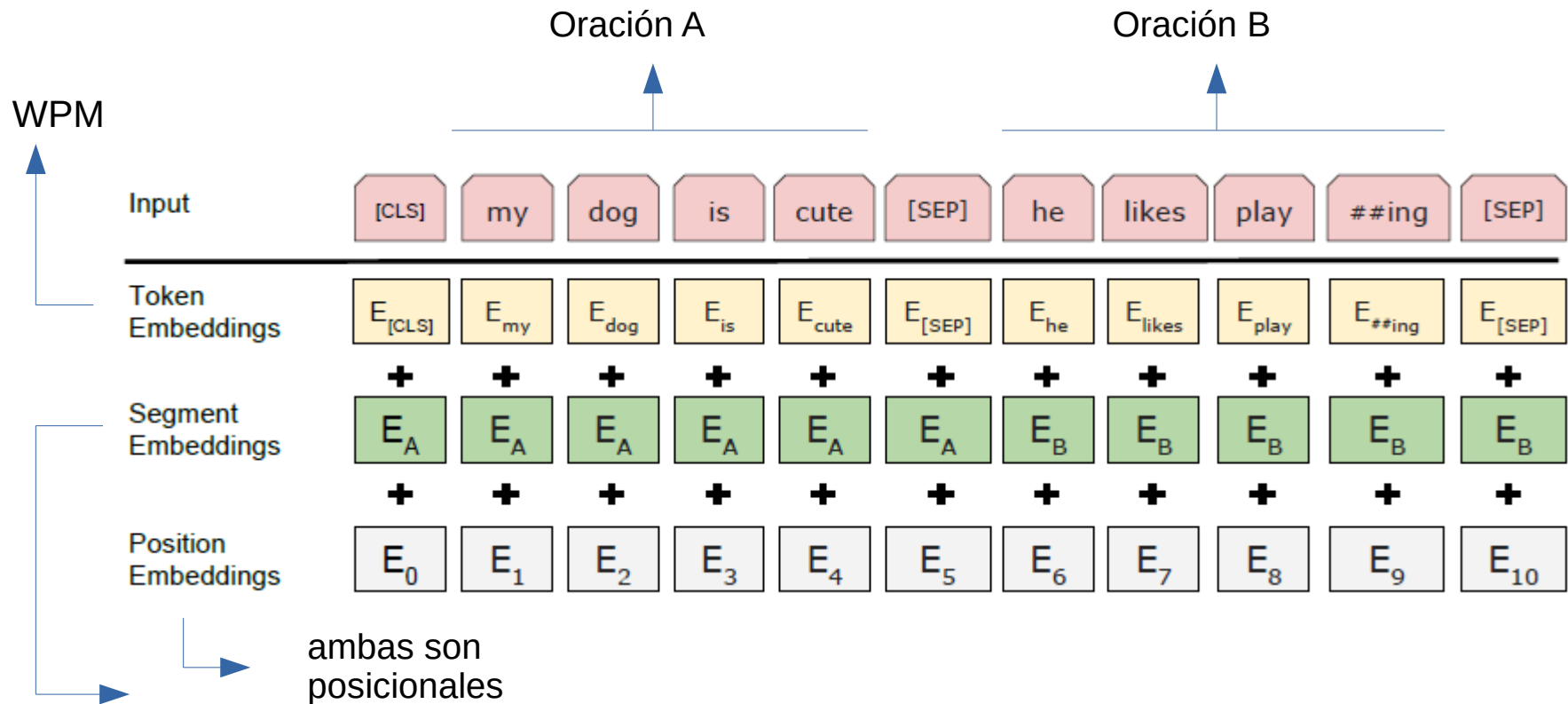
- Text2token usando el **Word-Piece Model** (igual que GPT-1 y GoogleMT)
- Task 1: **MLM** (*masked language model*) para *pre-training* a nivel de *tokens*
- Task 2: **Next sentence prediction (NSP)**



Jacob Devlin, [Ming-Wei Chang](#), [Kenton Lee](#), [Kristina Toutanova](#): BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. [NAACL-HLT \(1\)2019](#): 4171-4186

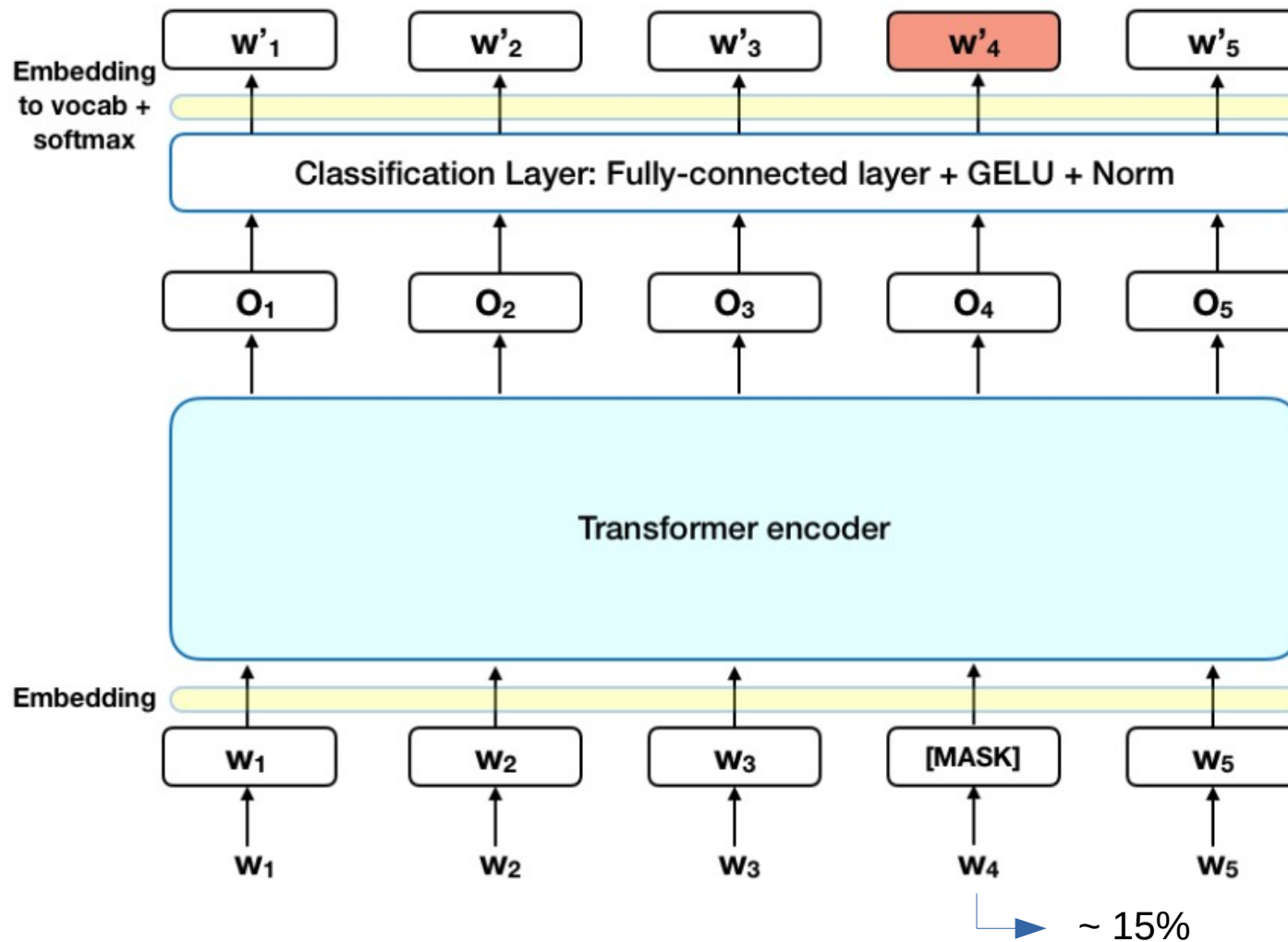
Bidirectional Encoder Representations on Transformers (BERT)

La entrada de **BERT**:



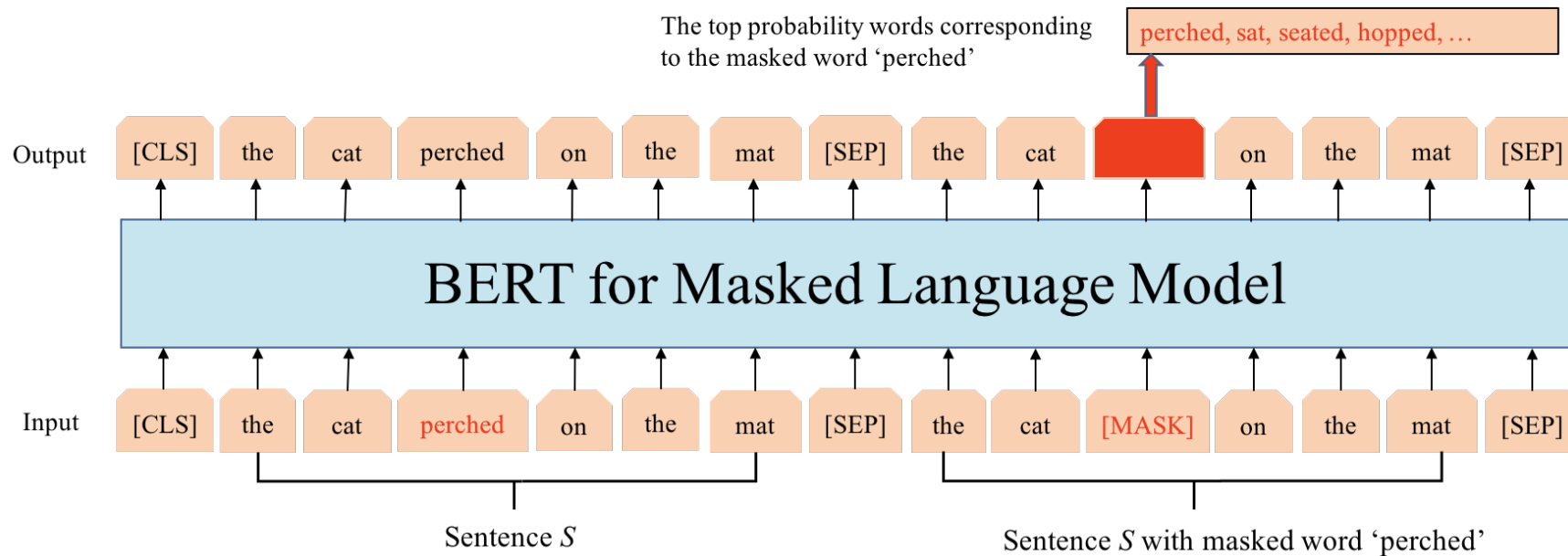
Bidirectional Encoder Representations on Transformers (BERT)

Masked Language Model:



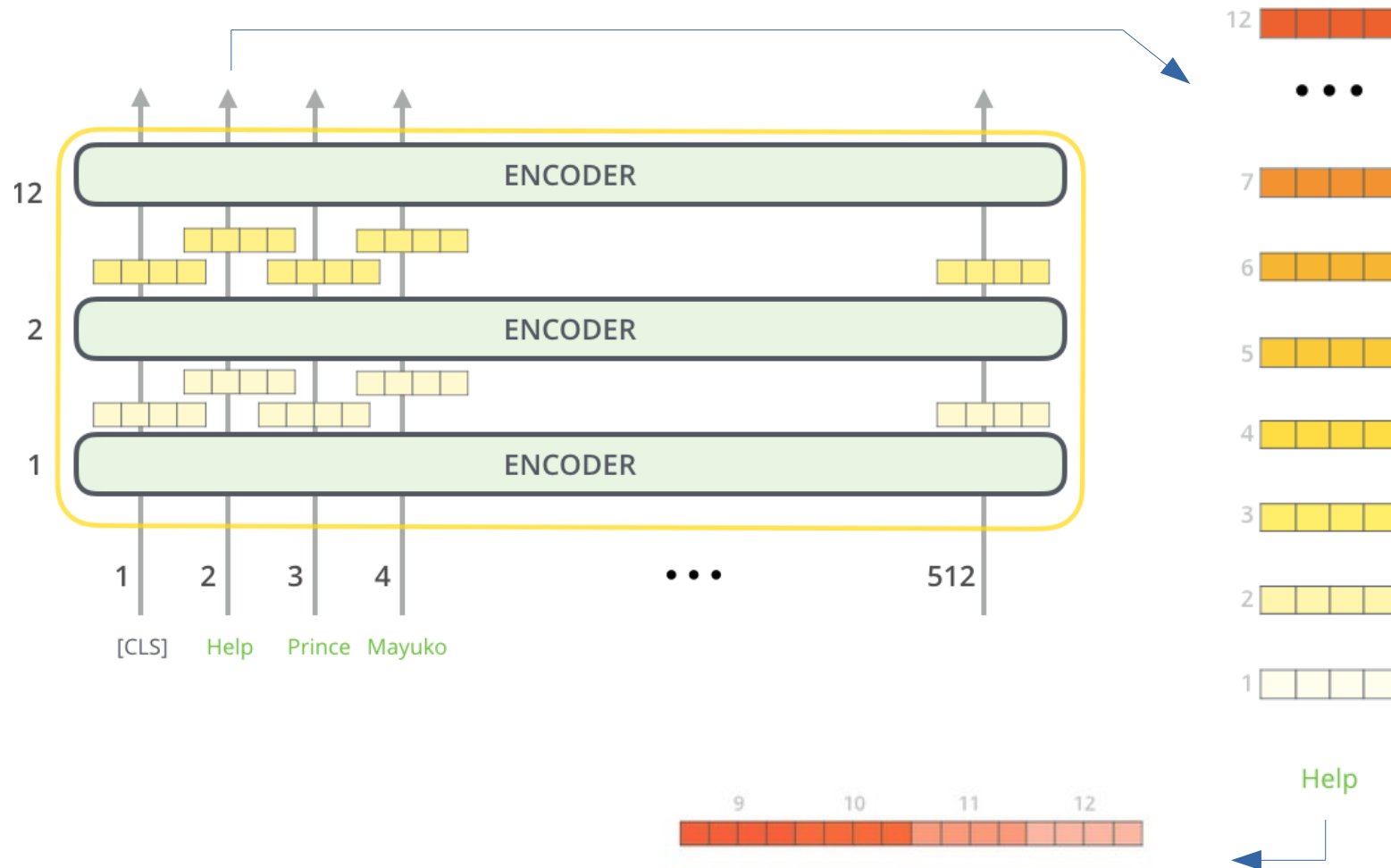
Bidirectional Encoder Representations on Transformers (BERT)

Masked Language Model:



Bidirectional Encoder Representations on Transformers (BERT)

BERT para vectorización de palabras (dependiente del contexto):



Bidirectional Encoder Representations on Transformers (BERT)

Aspectos prácticos

L: bloques de transformer, H: dim de h, A: # multi-heads

- BERT Base: L=12, H=768, A=12 (110M parámetros)
- BERT Large: L=24, H=1024, A=16 (340M parámetros)

Símbolos de BERT para representación de entrada (tokens especiales):

- [CLS]: primer token de cada sentencia
- [SEP]: separador de pares de sentencias

Pretraining: BooksCorpus (800M palabras) y Wikipedia (2,500M palabras)

Fine-tuning para downstream tasks: sobre el modelo preentrenado, se ingestan las entradas y salidas para la tarea específica.

Fine-tuning es menos costoso que pretraining. Por esta razón, BERT proporciona los modelos preentrenados.

Bidirectional Encoder Representations on Transformers (BERT)

Aspectos prácticos

Tareas de pre-entrenamiento

MLM (samplear un token de la sentencia de entrada):

- 80%: reemplazar el token por [MASK].
- 10%: reemplazar el token por un token at random.
- 10%: no reemplazar.

NSP (balance entre IsNext y NotNext)

- 50%: IsNext
- 50%: siguiente sentencia at random

- Training: batches de 256 (sentencias) * 512 tokens (128,000 tokens por batch); 1,000,000 steps (app 40 epochs sobre 3.3 billion word corpus). Implementado en tensor2tensor.

- Tiempo: BERT base en 16 TPU, BERT large en 64 TPU. 4 dias cada uno.